GeoLabeller

Interface Control Document

Software version v2.9.15

Generated 2026-10-28

This document specifies the external interfaces of GeoLabeller: the project file it reads and writes, the raster products it exports, the imagery it accepts, and the coordinate conventions all of these share. It is a description of the implemented behaviour, transcribed from the source.

1 Scope and Conventions

GeoLabeller is a desktop application for viewing georeferenced and non-georeferenced raster imagery and recording point annotations against it. This document specifies the interfaces through which other software exchanges data with it.

1.1 Interfaces covered

Interface	Direction	Section
Project file (.geoproj / JSON)	Read and written	3
Sub-image GeoTIFF export	Written	4.1
Optimized GeoTIFF export	Written	4.2
Waypoint file (GPX 1.1)	Written and read	3.9
Model review file (the model team's JSON)	Read, and its notes written	3.11
Label export file (KML 2.2)	Written	3.12
Candidates file (Find Candidates)	Written and read	3.13
Source imagery	Read	5

The HDF5 training-dataset export is specified separately and is not repeated here.

This document ships with the application: an installed copy sits beside the executable and opens from Help > ICD, so the version on hand always describes the build in front of you.

1.2 Notation

- Field types are JSON types: number, string, boolean, array, object.
- Pixel coordinates are (column, row) with the origin at the top-left pixel of the source image; column increases right, row increases down. Fractional values are meaningful and are not rounded on write.
- Geographic coordinates are WGS84 decimal degrees (EPSG:4326), longitude and latitude reported separately and never packed.
- Distances in metres are geodesic - measured on the WGS84 ellipsoid - not planar distances in a projected CRS. Before this they were computed on a sphere of mean radius, which reads short by 0.25% at 40 degrees latitude.

2 Coordinate Systems

Four coordinate systems appear across the interfaces. Only the first two are externally visible; the rest are described because they explain values that would otherwise look surprising.

System	Units	Where it appears
WGS84 (EPSG:4326)	degrees	Label lon/lat, waypoints, image corner coordinates
Source pixel	pixels	Label pixel_x / pixel_y, relative to the unmodified source image
Web Mercator (EPSG:3857)	metres	Internal display projection; not written to any file
Scene	metres	Internal canvas space; x = easting, y = -northing

2.1 Pixel coordinates are relative to the source image

Labels record the pixel they were placed on in the ORIGINAL image, at full resolution - not in the reprojected image shown on screen, and not at whatever pyramid level happened to be displayed. A label therefore addresses the same ground whatever the zoom was when it was placed, and a consumer can index the source raster directly.

2.2 Non-georeferenced imagery: the pixel zone

Imagery without a CRS cannot be placed geographically, so it is laid out in a reserved region of the internal scene beyond the valid extent of Web Mercator. This is an implementation detail with one external consequence: labels on such images carry pixel coordinates but no meaningful lon/lat, and those fields are written as 0.0.

Constant	Value	Meaning
PIXEL_ZONE_ORIGIN_X	25 000 000	Scene X at which the non-georeferenced region begins
PIXEL_ZONE_SCALE	50	Scene units per source pixel
PIXEL_ZONE_GROUP_GAP	5 000	Scene units between the columns of adjacent groups

2.3 Display resolution and memory

Imagery is displayed from pyramid overviews. The level chosen is the coarsest whose resolution is still no coarser than one screen pixel, subject to a cap on how much may be held at once. Images beyond that cap keep a coarse whole-image backdrop and read the detail for the visible area one tile at a time, so full resolution remains available at any zoom regardless of image size.

Constant	Value	Meaning
MAX_LEVEL_PIXELS	150 000 000	Largest whole-image level held in memory
BACKDROP_MAX_PIXELS	4 000 000	Cap for the backdrop behind windowed detail tiles
TILE_SIZE	512	Edge of a display tile, in destination pixels

3 Project File

A project is a single UTF-8 JSON document holding the class list, every image referenced, the labels on them, and any waypoints. Images are referenced by absolute path; their pixels are never copied into the project.

3.1 Top level

Field	Type	Description
version	string	Format version of this document, currently "5.3". Governs how label coordinates are interpreted - see 3.4.
classes	array of string	Ordered class names. A label's class must appear here.
images	array of object	One entry per referenced image; see 3.2.
waypoints	array of object	Named geographic bookmarks; see 3.5. Absent before version 3.3.
descriptions	array of string	Preset description strings for the active-description picker (applied to newly placed labels; a label's own description stays free text). Present only when non-empty; absent before version 4.2.
mask_names	array of string	Preset mask names offered by the Snippet Editor when a mask is added. A name typed there joins the list. Editing it never renames a mask already painted - mask names on labels (3.3) are independent. Present only when non-empty; absent before version 4.4.
display_settings	object	How each group's imagery is drawn on screen, keyed by project group path; see 3.10. Only groups that set their own appear; a sub-group without an entry uses its nearest ancestor's. Display only - no export reads it. Present only when non-empty; absent before version 4.6.
model_review	array of object	Model Review's memory: one record per chip of the model team's mistakes that a person gave a verdict on; see 3.11. Present only when non-empty; absent before version 5.0.
_next_id	number	Next label id to allocate. Label ids are unique within a project and never handed out twice: undo and Clear All Labels leave the counter where it is, so anything naming a label by id - a review file's notes - cannot land on a later label (2.9.11).
_next_waypoint_id	number	Next waypoint id to allocate.

3.2 Image object

Field	Type	Description
path	string	Absolute path to the image file, and the key that identifies it: one entry per file. Separators are normalised on load and on every add (".." and doubled separators resolved; on Windows the separator is written as "/"), so the same raster reached through different spellings is one image rather than two. A file listed twice under two spellings has its labels merged on load. A path that does not exist is first re-tried against the project file's own folder by matching progressively longer path tails, so a project that travels with its imagery opens unchanged on another machine; anything still missing can be relocated interactively (File > Locate Missing Images), or taken out of the project with its labels (File > Remove Missing Images). Until one or the other is done, a missing image's entry and labels stay in the project and in every ground truth export.
name	string	File name without extension.
group	string	Group hierarchy, '/' separated; empty when ungrouped.
labels	array of object	Labels on this image; see 3.3.
original_width	number	Source image width in pixels. Required to interpret label coordinates - see 3.4.

Field	Type	Description
original_height	number	Source image height in pixels.
reader	object	Extension to reader name, e.g. {"tif": "default"}.
affine_coeffs	array of number	Six coefficients of the source affine transform. Omitted when unknown.
crs_epsg	number	EPSG code of the source CRS. Omitted for non-georeferenced imagery, and for a CRS that has no EPSG code - see crs_wkt.
crs_wkt	string	WKT of the source CRS, written only when the CRS has no EPSG code (a local grid, an ESRI-defined projection, a custom datum). Exactly one of crs_epsg / crs_wkt is present on a georeferenced image. Absent before version 4.3.
corners_wgs84	object	Convenience only: top_left / top_right / bottom_right / bottom_left, each {lat, lon}. Written when the transform and CRS are known; derived, never read back.
geodesic_width_m	number	Convenience only: ground width along the top edge, metres.
geodesic_height_m	number	Convenience only: ground height along the left edge, metres.
hard_negative_source	boolean	The image contains confusers but no true positives; the H5 export can opt in to sliding it whole into hard negatives. Present only when true; absent before version 3.5. Kept even while the image is not loaded, like labels.
location	string	Free-text location tag, e.g. "New York Area". Stored per image but normally applied to a whole group at once (right-click a group > Set Location...), and carried into the HDF5 export as the per-sample locations dataset. Present only when non-empty; absent before version 4.1.

3.3 Label object

Field	Type	Description
id	number	Sequential id, unique within the project.
unique_id	string	UUID v4, unique to this label.
class_name	string	Must appear in the top-level classes.
pixel_x	number	Column in the source image. Stored as a FRACTION of original_width from version 2.1 - see 3.4.
pixel_y	number	Row, stored the same way.
lon	number	WGS84 longitude, or 0.0 for non-georeferenced imagery.
lat	number	WGS84 latitude, or 0.0.
object_id	string	UUID v4 shared by every label of the same real-world object. Labels are linked by sharing this value. Every gt=true sample of the HDF5 training export records it in the per-sample object_ids dataset (aligned with images; "" for a hard negative, which is a view of no object), so the views of one object can be grouped, held out of a split together, or recognised as already present before appending them again.
length_m	number	Measured object length in metres. Omitted when unmeasured.
width_m	number	Measured width in metres. Omitted likewise.
description	string	Free text describing this labelled object. Belongs to this label alone and is never copied to labels sharing its object_id. Omitted when empty; absent before version 3.4.
orientation_px_rad	number	Unit-circle principal angle of the object's drawn orientation in UN-WARPED source-pixel space: radians in (-pi, pi], x along the rows (+column), y up the columns. A vector from top-right to bottom-left is about -3pi/4 (-135 degrees); rightward along a row is 0; up the image is +pi/2. Omitted when never drawn; absent before 3.7.

Field	Type	Description
orientation_deg	number	The same drawn vector as a TRUE-north heading: degrees clockwise from north in [0, 360), from a WGS84 geodesic azimuth of the endpoints through the image's transform - not grid north. Omitted for non-georeferenced imagery or when never drawn. Per label even when linked: two views of one object are two measurements.
orientation_derived	boolean	Present (true) only when this orientation was NOT drawn by hand but propagated from a linked label: the group's shared true-north heading, re-derived into this image's own pixel space through its georeferencing. Cleared when the user draws over it. Absent before 3.8.
confidence	integer	How sure the labeller is of this label, for training: 1 (lowest) to 5 (highest), set in the Snippet Editor. Omitted when unrated. Readers must treat anything other than a whole number 1-5 as unrated - GeoLabeller loads it so. Per label, like the orientation. Absent before 4.5.
review	string	The label's review in the Snippet Editor: "accepted", "rejected" or "recheck" (to be looked at again). Omitted when unreviewed; anything else reads as unreviewed. A rejected label stays in the project and is left out of the HDF5 and sub-image exports. Absent before 4.7.
attribution	object	Who did what, keyed by what was done: created, class, description, group_id, link, orientation, size, confidence, review, masks, position. Each value is {by, at}: the user name entered in GeoLabeller (File > Switch User; a name, not a login) and the UTC time, ISO 8601. created is who placed the label; the others are who last set that value. Each mask entry carries the same by and at for whoever drew or last repainted it. Omitted when empty - labels made before 4.8, or with no user name set. In the project and ground truth only; the HDF5 export does not carry it.
masks	array of object	Named binary masks painted on this label's snippet in the mask editor, for measuring the object's pixel distribution against the background. Each entry is {name, x0, y0, width, height, rle}; the full encoding is specified in 3.8. From version 4.0 the window is the full source image. Present only when at least one mask exists; absent before 3.9.
group_id	string	Human-readable name for the linked-object group - the readable companion to object_id. The inverse contract to description: it SHALL be identical across every label sharing an object_id. Updating it anywhere updates the whole group; a file that violates the invariant is repaired on load (first non-empty name in the group wins). Omitted when empty; absent before 3.6.
geodesic_distance	number	Convenience only: metres from the image's left edge at the label's row. Derived, never read back.

3.4 Label coordinate storage (important)

From version 2.1, pixel_x and pixel_y are stored as FRACTIONS of the image dimensions, not as absolute pixels. A consumer must multiply by the image size to recover pixel coordinates:

```
pixel_x_absolute = pixel_x * original_width
pixel_y_absolute = pixel_y * original_height
```

The conversion applies only when the document version is 2.1 or later AND original_width and original_height are both greater than zero. If either is zero the values are absolute pixels and must be used unchanged. Both the writer and the reader apply this rule, so a file is self-consistent; it matters when reading a project with anything other than GeoLabeller itself.

3.5 Waypoint object

Waypoints are named geographic bookmarks belonging to the project rather than to any image. They are navigation aids and are never exported as training data.

Field	Type	Description
id	number	Sequential id, unique within the project.
name	string	Display name, e.g. "WP 1".
lat	number	WGS84 latitude in degrees.
lon	number	WGS84 longitude in degrees.

3.6 Version history

Version	Change
1.0	Label-centric layout: a flat labels array, each entry naming its image. Still readable.
2.0	Image-centric layout: images each own their labels.
2.1	Label pixel coordinates became fractions of the image dimensions (see 3.4).
3.2	Adds derived convenience fields: corner coordinates and geodesic extents.
3.3	Adds the top-level waypoints array and _next_waypoint_id. Files written before this simply have none.
3.4	Adds the optional per-label description string.
3.5	Adds the optional per-image hard_negative_source flag.
3.6	Adds the shared per-group group_id string on labels.
3.7	Adds the optional orientation_px_rad and orientation_deg per label.
3.8	Adds the optional orientation_derived flag for orientations propagated from a linked label.
3.9	Adds the optional per-label masks array of named, run-length-encoded binary snippet masks (see 3.8).
4.0	Mask entries are written anchored to the FULL source image (x0 = y0 = 0, width/height = the image's) instead of the paint-time snippet window. The schema is unchanged and readers honor whichever window an entry records; 3.9-windowed entries are re-anchored on load. Numbered 4.0 rather than 3.10 because versions compare as strings and "3.10" would sort below "3.9".
4.1	Adds the optional per-image location string - a free-text location tag, normally applied to a whole group at once.
4.2	Adds the optional top-level descriptions array: preset strings for the active-description picker. Labels are unchanged.
4.3	Adds the optional per-image crs_wkt, carrying the source CRS of an image whose CRS has no EPSG code. Such images previously recorded an affine transform with no CRS at all.
4.4	Adds the optional top-level mask_names array: preset names for the mask editor's picker. Masks themselves are unchanged.
4.5	Adds the optional per-label confidence rating, 1-5. Nothing else changes.
4.6	Adds the optional top-level display_settings object (3.10). Nothing else changes.
4.7	Adds the optional per-label review string (3.3). Nothing else changes.
4.8	Adds the optional per-label attribution object and the by / at of each mask (3.3). Nothing else changes.
4.9	Adds the optional stretch object to a display_settings entry (3.10). Nothing else changes.
5.0	Adds the optional top-level model_review array (3.11). Nothing else changes; the major number only because minors are single digits.
5.1	Adds the optional palette flag to a display_settings entry (3.10). Nothing else changes.
5.2	Adds the optional group string to a model_review record (3.11). Nothing else changes.
5.3	The top-level model_review array is no longer written: the review file carries its own notes (3.11). A 5.0 to 5.2 document's array is still read, once, to move its notes into the review file as that is opened; it goes at the next save. Nothing else changes.

Readers should compare the version as a string and treat anything at or above a threshold as supporting that feature. Older documents load unchanged; the only migration is the 4.0 mask re-anchoring, which the reader performs structurally on load.

3.7 Example

A project with one image, one label and one waypoint. Note the fractional pixel coordinates: label 7 sits at absolute pixel (2048, 1024), being 0.5 x 4096 and 0.25 x 4096.

```
{
  "version": "3.4",
  "classes": ["vehicle", "building"],
  "images": [
    {
      "path": "D:/imagery/tile_014.tif",
      "name": "tile_014", "group": "survey/north",
      "original_width": 4096, "original_height": 4096,
      "reader": {"tif": "default"}, "crs_epsg": 32618,
      "labels": [
        {"id": 7, "class_name": "vehicle",
         "pixel_x": 0.5, "pixel_y": 0.25,
         "lon": -73.9832, "lat": 40.7536,
         "unique_id": "3b1e...", "object_id": "8f2c...",
         "length_m": 4.6, "width_m": 1.8,
         "description": "white pickup, tarpaulin over the bed"}
      ]
    }
  ],
  "waypoints": [{"id": 1, "name": "WP 1",
                 "lat": 40.7536, "lon": -73.9832}],
  "_next_id": 8, "_next_waypoint_id": 2
}
```

3.8 Snippet masks and their encoding

A mask marks which pixels of a label's snippet belong to the actual object. Masks are stored per label as independent named binary layers - several may coexist and OVERLAP on one snippet - and are painted in the Snippet Editor (Labels > Snippet Editor). It keeps masks exclusive unless Allow Overlap is switched on, but that is an editing aid, not a guarantee of the format: readers must still expect overlapping masks, from projects painted with it on and from projects older than it.

Anchoring. x0/y0/width/height describe the mask's window in SOURCE-image pixels. From format 4.0 the window written IS the full source image - x0 and y0 are 0 and width/height equal the image dimensions - so the decoded rle is co-registered with the imagery: mask pixel (row, col) is image pixel (row, col), with no anchoring arithmetic. Format 3.9 wrote the snippet crop at the moment the mask was painted instead (per the centred-window rule of 4.1). Readers SHALL honor whichever window an entry records - a mask re-projects into any other window, a different snippet size or an export crop, by pure translation and clipping - and SHOULD stream the runs rather than decode the full window, which for a 4.0 entry over survey imagery can cover billions of pixels. Only the runs bordering unpainted ground grow with the image; the run COUNT is essentially the snippet-window count.

Compression. The rle field run-length encodes the binary window read row-major (row 0 left to right, then row 1, and so on). Entries alternate between runs of 0s and runs of 1s, and the list ALWAYS begins with a run of 0s - a mask whose very first pixel is set therefore begins [0, n, ...]. The runs MUST sum to exactly width x height; readers SHALL reject any mask where they do not, rather than render a shifted mask. Worked example: the 4x3 window

```
0 0 1 1
0 0 1 1
0 0 0 0

encodes as  rle = "2,2,2,2,4"
(2 zeros, 2 ones, 2 zeros, 2 ones, 4 zeros = 12 pixels)
```

A painted blob of a few thousand pixels typically encodes to a few hundred integers, which is why masks can live inside the project JSON and share its save/autosave/recovery behaviour rather than requiring a sidecar file.

Delivery (informative). The HDF5 training export - specified separately, see 1.1 - carries the painted masks in three aligned datasets: mask_pixels (M x H x W, uint8 binary), mask_sample (M, the images row each mask belongs to) and mask_names (M, UTF-8). Every example crop receives every mask that intersects it, re-anchored to that crop's window - so in the HDF5 there is no x0/y0 to apply: mask_pixels[j] overlays images[mask_sample[j]] pixel for pixel as-is. Extracting them needs nothing beyond h5py:

```
with h5py.File(path) as f:
    for j in range(f['mask_pixels'].shape[0]):
        image = f['images'][f['mask_sample'][j]]
        mask  = f['mask_pixels'][j].astype(bool)
        name  = f['mask_names'][j].decode()
        object_pixels = image[mask]
```

Delivery (informative), confidence and orientation. From 2.0 the HDF5 training export also carries, aligned with images row for row: confidence (uint8; the label's 1-5 rating, 0 when unrated and for every hard negative), orientation_px_rad and orientation_deg (float32; the label's pixel angle and true-north heading as defined in 3.3, NaN where there is none - 0 is an angle, so it cannot mean none) and orientation_derived (bool; propagated from a linked label). The pixel angle holds in each crop as stored: a crop is a plain window of the source raster, which only translates it. Every crop of a label carries that label's values. Each group is written only when some exported sample has a value for it - an export with nothing to say adds no such dataset - and rows written before a group first appears read as unrated / no orientation. From 2.9.15 a sample also carries nodata_filled (uint32): how many of its pixels held no data in the source (NaN or the nodata value) and were filled in - with the window's own mean in the float32 format, black in uint8 - 0 for a sample cut whole from real ground; written, like the groups above, only when some sample has a count.

3.9 Waypoint exchange file (GPX)

Export > Waypoints (GPX)... writes every waypoint of the project to a GPX 1.1 file (UTF-8 XML, default namespace http://www.topografix.com/GPX/1/1), the interchange format GPS receivers, Google Earth, QGIS and GDAL read. The root gpx element carries version="1.1" and creator="GeoLabeller <version>"; metadata/time is the UTC time of writing. Each waypoint is one wpt element with lat and lon attributes (WGS84 decimal degrees, eight decimals) and a name child. Nothing else is written: waypoints carry no elevation, time or description.

File > Import Waypoints... reads a GPX file from any producer - GPX 1.0 or 1.1, with or without a namespace - taking every wpt element's lat, lon and name; rtept and trkpt are not waypoints and are ignored, as is any other child. It also reads the waypoints array of a GeoLabeller project file (3.5), so a colleague's waypoints can be taken from their project directly (Import Ground Truth leaves waypoints alone). An entry without a usable position - missing, non-numeric or out-of-range lat or lon - is skipped and counted. Only what is new is added: an entry whose position is within 1e-7 degrees of an existing waypoint, and whose name matches it (case and padding aside) or is blank, is already in the project. New waypoints take fresh ids; a blank name becomes "WP n". A confirmation states the counts before anything changes, and the project is marked unsaved afterwards.

3.10 Display settings

The display_settings object maps a project group path to how that group's imagery is drawn on the canvas and in snippet views. It changes nothing that is measured, stored per label or exported: the HDF5 dataset and the sub-image export read the raw bands.

Field	Type	Description
mode	string	"single": one band drawn as grayscale. "rgb": a band each for red, green and blue.
band	number	single mode: the 1-based band drawn. A band the file lacks falls back to its last band.
bands	array of number	rgb mode, optional: the 1-based bands for red, green and blue. Absent means the file's first three bands, or its only band three times - what is drawn without any settings.
channels	array of object	The adjustment of each displayed channel: one entry in single mode, three (red, green, blue) in rgb mode. Each has brightness (-100..100), contrast (-100..100) and gamma (0.2..5.0), and in rgb mode optionally hidden: true, which draws that channel as exactly 0 whatever the other three say.
stretch	object	Optional (absent before 4.9): how 16-bit and floating-point values become display bytes before the channel adjustments. kind "percentile" with low and high percentiles (0..100, to a tenth), each band stretched between those percentiles of its own sampled values; or kind "range" with low and high values, the same for every band, drawn as 0 and 255 - 0 and 1 for imagery that is already a fraction. Values outside the window clip. Absent means percentiles 2 to 98, how such imagery was always drawn. 8-bit imagery is drawn as stored whatever this says.
palette	boolean	Optional (absent before 5.1). A band that carries an embedded colour table (GDAL's palette: a colour per pixel value) is drawn in those colours when it is the band drawn in single mode, or a one-band file's only band under the default settings; the stretch does not apply, the channel adjustments still do. false draws the values as grey instead. Absent or true means through the table.

Adjustments work on the 8-bit display value v of a band - after the stretch above, for 16-bit and float imagery - in this order: v + 1.27 x brightness, then contrast as a gain of 8^(contrast/100) around 128, clipped to 0..255, then 255 x (v/255)^(1/gamma). Gamma above 1 lifts the mid-tones.

3.11 Model review: the mistakes file and its notes

Model Review (Labels > Model Review...) reads a file written by the model team listing the chips their classifier was confidently wrong about - clutter it took for a real class - for a person to label or ignore, and imports the verdicts as labels. The file is a UTF-8 JSON array of records, or an object holding one under "chips" or "items":

Field	Type	Description
image	string	The image's file name, in any directory or none ("file", "filename" and "path" are read too). Matched to the project's images by name, case-insensitively; a name the project holds twice is decided by the longest shared folder tail when the record gives a directory, and rejected otherwise. A record whose image is not in the project is rejected; the rest of the file still loads.
image_width, image_height	number	The size of the image the centre was measured on ("width" / "height" are read too). Optional: without them the centre is taken at the project image's own size. With them, a project image of another size is the same picture at another resolution, and the centre is scaled by the ratio of the sizes. Fractions of the image work by the same rule: a size of 1 by 1 and x, y in 0..1.
x, y	number	The chip's centre, decimal pixels at image_width by image_height; column then row ("pixel_x" / "pixel_y" are read too). Kept inside the image.
class	string	What the model called it ("predicted" / "class_name" are read too). A record whose class is not one of the project's classes is dropped - except "candidate", a detector's (3.13).
score	number	How sure the model was, 0..1 ("confidence" / "probability" are read too). Chips are shown from highest to lowest; 0 when absent.

The chip's size is not in the file: the window reads a square of the user's chosen size around each centre, as the Snippet Editor does. An imported label is an ordinary label (3.3) at the chip's centre on its image, with lon/lat through the image's georeferencing, attribution for the current user, and a description of the form "model: <class> <score>, <file>".

The memory of what was decided is the file's own (2.9.4): an import writes a review object onto each chip's record, with every other field of the record and of the file as it was, atomically (a temporary file replaces the original). A record with a review is reviewed: kept off the grid unless Show reviewed, and given a new verdict its label is changed, removed for "ignore", or made. The labels are the truth: a review naming a label that is not in the project, on another image, or no longer of the class the review says it became (given another by hand, or an import onto it undone) does not stand, and the chip is plain again - and already in project by that label, where it lies within the distance. Combine Projects renumbers the second project's labels, and that project's review files still name labels by their old ids: their notes stand only where the id still lands on a label of the right class on the right image. A file that cannot be written is reported once, as is one whose records are no longer the chips they were (replaced since it was opened: a note goes only onto a record of the chip's image); the notes hold for the session. Nothing of this is written to the project (the 5.0 to 5.2 model_review array is no longer written; one found in a project is moved into a review file as that file is opened, matched by image name and centre within Same chip within, each entry to one chip).

Field	Type	Description
became	string	The verdict: a class name, or "ignore" for a chip looked at and not labelled.
label_id	number	The id, in the project, of the label the verdict made or took over. Absent for an ignore.
by, at	string	Who gave the verdict, and when (UTC, "2026-10-27T10:00:00Z"); absent when no user name was set.

A chip whose centre lies within the window's Labelled within distance (the Import Options distance to start with) of a label already on its image - not the label its own review names - is already in project: shown apart, on the grid unless Show already in project is unticked, the labels marked in the Details panel. Its verdict goes onto the nearest of those labels, which takes the class, unless Add as a new label here is ticked for it, when a new label is made beside and spared the duplicate rule; "ignore" leaves the label alone. The comparison is made again whenever the project's labels change.

3.12 Label export file (KML)

Export > Labels (KML)... writes every label that has a position on the ground to a KML 2.2 file (UTF-8 XML, default namespace http://www.opengis.net/kml/2.2), which Google Earth, QGIS, ArcGIS and GDAL read. The Document is named after the project file and carries a Style per class (id class-<n>, n the class's place in the project's class list; IconStyle/color the class's marker colour as aabbggrr, opaque; a circle icon) and a Folder per class holding its Placemarks, in label id order. A Placemark's id is label-<id>; its name is the class; its description the label's own, when there is one; its TimeStamp/when the creation time, when a user name was set; its styleUrl the class's Style; and its Point/coordinates lon,lat,0 in WGS84 decimal degrees, eight decimals. ExtendedData carries Data name/value pairs: class, image (the file name), group, pixel_x, pixel_y (two decimals), label_id, unique_id, object_id, and when set length_m, width_m, heading_deg (true north), confidence (1-5), review, created_by, created_at and description. Left out, and counted on the status line: a label on imagery without georeferencing and with no recorded position, and a label rejected in review (as the HDF5 and sub-image exports leave it out). Nothing reads KML back: the project file (section 3) is the round trip.

3.13 Candidates file (Find Candidates)

Labels > Find Candidates... runs a detector (app/detect: local contrast against a robust block background, or a difference of Gaussians over a size range) over the chosen images and writes what it found to a file beside the project file, named after it with the suffix .candidates.json, overwritten by each run. The file is a Model Review file (3.11): a UTF-8 JSON object whose "chips" array holds one record per candidate, with the fields of 3.11 - image (the project's path to it, so same-named images in different groups are told apart), image_width and image_height

(the raster's), x and y (the candidate's middle, full-resolution pixels), class (always "candidate") and score (0..1, the strength s in robust sigmas as s / (s + 8)) - and three more: strength (the sigmas themselves), size_px (the patch's diameter as a disc, in source pixels) and polarity ("bright" or "dark"). The object also records created (UTC), creator, detector (its key), settings (the run's, as the dialog remembers them) and summary (the run's counts). Model Review reads the file like any other and takes the class "candidate" without it being one of the project's; its memory entries carry predicted "candidate" and the run as their source. A candidate within the memory's tolerance of an entry, or within the dialog's distance of a label, is not written.

4 Raster Exports

4.1 Sub-image export

Writes one GeoTIFF per label, cropped around it. The requested size is given in ground metres and converted to pixels using the true resolution at the label, so the crop covers the requested ground area whatever the CRS and whatever the pixel aspect.

Property	Behaviour
Output path	<output>/<class_name>/<object_id>_<label_id padded to 6>.tif
Window	Centred on the label's pixel, shifted (never cropped) to stay inside the image, so edge labels still yield a full-size crop.
Pixels	Copied unmodified: all source bands, original data type, no rescaling.
Georeferencing	Source CRS preserved; transform adjusted to the window.
Other metadata	nodata and per-band colour interpretation preserved.
Compression	LZW.
Skipped	Labels on imagery with no CRS, labels outside the image, and windows that would be under half the requested size.

4.2 Optimized GeoTIFF export

Rewrites imagery as tiled, pyramided GeoTIFFs for fast display. Pixel values, band count, data type, CRS and transform are all preserved - only the storage layout changes.

Property	Behaviour
Output path	Mirrors the group hierarchy under the output directory; keeps the source stem with a .tif extension.
Layout	Tiled, 512 x 512 blocks by default.
Overviews	Built to the requested decimation factors. Average resampling by default; paletted rasters always use nearest, so palette indices are never blended into invalid values.
Compression	DEFLATE by default; may be set to another codec, none, or KEEP to retain the source's. A predictor is chosen to suit the data type.
Large files	BIGTIFF written when the result may exceed 4 GB.
Existing files	Skipped unless overwrite is requested. Output is written to a temporary file and moved into place, so an interrupted export leaves no partial .tif.

5 Source Imagery

Imagery is read through GDAL, so any format GDAL can open may be loaded; file dialogs filter to .tif and .tiff by default, and directory import scans for those extensions.

5.1 Georeferenced and non-georeferenced imagery

An image with a CRS is reprojected for display and its labels carry both pixel and WGS84 coordinates. An image without one is shown at its own pixel scale in the region described in 2.2, and its labels carry pixel coordinates only, with lon and lat written as 0.0.

5.2 Recommendations

- Supply tiled GeoTIFFs with overviews. Without a pyramid the whole image must be decoded before anything can be shown; with one, a coarse level appears immediately and sharpens. Section 4.2 produces suitable files.
- 8-bit imagery is displayed exactly as stored. Other data types are linearly stretched for display: by default between the 2nd and 98th percentile of a sampled read, or as the group's Display Settings say (other percentiles, or a fixed range of values such as 0 to 1; see 3.10). The underlying values are never modified.
- Very large mosaics are supported: detail for the visible area is read a tile at a time, so display cost follows the viewport rather than the size of the image.
- Reads are threaded. Reprojection into the scene's projection, which is most of what displaying an image costs, and GDAL's own decoding each use about a quarter of the machine's cores, at most four. GDAL_NUM_THREADS is set only when the environment does not already define it, so an administrator's value wins.
- GDAL sidecar files - an external .ovr pyramid, an .aux.xml, a world file - are honoured. A folder holding none is opened without GDAL's per-open listing of the directory, which is the expensive part of an open on a network share. That is decided once per folder per session, so a sidecar added while GeoLabeller is running is picked up at the next start.

6 Operation

This section is included because the way a label is produced decides which fields it carries. A consumer reading a project file will meet labels that share an object_id, labels with measurements and labels whose lon/lat are zero, and 6.3 explains where each comes from.

6.1 Interaction modes

The canvas is always in exactly one mode. Modes differ in what a left click does and how the view is navigated; none of them change how a label is stored.

Mode	Left click	Purpose
Pan	Drags the view	Navigating. Right click also offers waypoint and layer actions.
Label	Places a label	Ordinary labelling of whatever is on screen.
Cycle	Places a label	Steps through a group one image at a time; Space advances, Ctrl+Space goes back. At either end the image stays showing and a short notice says so; C there starts the group again.
View Cycle	Places a label	As Cycle, but only over the selected group's images that intersect the current view, in tree order; the status bar shows how many of the group are on screen. A detour (Ruler, Pan, Label) resumes while the image it was on is still on screen. At the end, V runs the same images again, or takes what is on screen once the view has moved. Only an image the cycle switched on is hidden again.
Labeled Cycle	Places a label	As Cycle, over only the selected group's images that have labels (toolbar, or Shift+C), in the same order with the same keys; the status bar shows how many of the group are labeled. The queue is what is labeled when the run starts. A detour resumes on the image it left while that image is still labeled.
Waterfall	Places a label	Stacks a bottom-level group vertically as raw pixels and glides through it (hold Space; the glide and panning stop at the ends of the stack); every label in the group stays visible. It opens on the image a cycle was on, else at the bottom of the stack. Every image of the group is switched on while stacked; leaving switches off those the mode switched on, returns the view to the geography of the image being examined, and a cycle started next opens on that image. Leaving for a mode with no image of its own (Pan, Label, Ruler) keeps that one image on. W with another bottom-level group selected stacks that group instead.
Ruler	Drags a measurement	Reading ground distances. Records nothing in the project.

Two overlays work on top of whichever labelling mode is active, rather than replacing it:

Overlay	Effect
Chain Link	Clicking labels one after another links them into a single object; each click writes the shared object_id immediately. Right-clicking a label also offers Link by Box: drag a desktop-style box and every label inside is linked to that label's object. Linked groups always wear a coloured halo ring - one colour per group, distinct between groups, updated the moment links change.
Size	In the Snippet Editor's Single view, the Measure tool draws a label's length and then its width on its snippet, writing length_m and width_m; each is stored as it is drawn, either can be redrawn, and right-click clears both. The line's two ends go through the image's own georeferencing to WGS84 and are measured on the ellipsoid, so an image with no georeferencing cannot be measured. With 'Apply size to linked labels' on, every label sharing the object_id receives the same values; off, the default, each view is measured on its own. Until 2.1 this was done on the map canvas (right click a label, or M), which now keeps only the Ruler - a distance that touches no label.

Overlay	Effect
Description	Right click a label to write a free-text note on it, shown as the marker's tooltip. Stays with that label when it is linked. A project may also carry preset descriptions (3.1); the active preset - toolbar picker, or Shift+1-9, pressing it again or Shift+0 for none - is applied to every newly placed label, and editing the presets never changes labels already written.
Locate missing images	File menu. Rebinds a shared project's images to their paths on this machine: search a folder (filename matches verified against each image's recorded size and CRS) or locate one file and apply its old-versus-new path prefix to the rest. Applied paths persist when the project is saved.
Recent projects and folders	File > Open Recent lists the projects opened or saved lately, newest first, and opens one directly; an entry whose file has moved says so and drops off the list. Each kind of file dialog also reopens where it was last used - imagery, project files and exports are remembered apart. Both are per user (they are how somebody works, not what the work is), so neither travels with a project file.
Import ground truth	File menu. Adds the labels from a GT file (Export > Ground Truth, or any project file) made on another machine to the current project, whose images were loaded here - typically with Add Directory. Each GT image is matched to a loaded image by filename (case-insensitive, read by the writing OS's path rules), verified against the size and CRS the GT recorded; several fits go to the longest shared folder tail, and a tie matches nothing. Labels keep their pixel positions, fields, unique_id and object_id, and take new ids; one whose unique_id is already in the project is skipped, so importing again adds only what is new. Their classes and the GT's presets are added; a hard-negative flag or location tag is taken only where the local image has none; waypoints are not imported. A preview lists each GT image's outcome before anything changes, and labels on images that cannot be matched are left out.
Import ground truth at another resolution	Ground truth made on downsampled copies of the imagery can be applied to the originals: load the full-resolution folder, then import the GT. A loaded image of the same name but another size is taken to be the GT's image at another resolution. Nothing checks that the two cover the same ground: the folder loaded must be a mirror of the GT's imagery, same names and same extents, and that is the user's to ensure. Only a different CRS still makes it a different file. Where an image of the GT's own size is also loaded under that name, it is preferred. Each label is placed at the same ground position (the GT's pixel through the GT image's transform, then this image's), or at the same fraction of the image where georeferencing is missing, and kept inside the image; its lon/lat follows. Positions are taken as given: a label is accurate to about half a pixel of the coarser image. Masks, being the other size's pixels, are left out and must be redrawn. Everything else arrives unchanged; an orientation's pixel angle is adjusted only when the two axes scale differently. The preview marks these images Found, other resolution, with both sizes and the masks to be left out.

Overlay	Effect
Model Review	Labels > Model Review... opens a window for the model team's file of mistakes (3.11): the chips their classifier was confidently wrong about, matched to the project's images by file name and shown one predicted class at a time from the highest score down, cut at a chip size of the user's choosing. A click gives a chip the active label - a project class, one added from the window, or Ignore - a Shift-click a run of them, a Ctrl-click picks several for a number key; or, from the keyboard, the arrow keys move a cursor and a number key or Enter labels the chip under it and moves on, I ignoring the rest of the page. Ctrl+Z takes back the last thing done in the window before reaching the project's history. A double-click puts the chip's centre - where the label will go - on the object without labelling it, and the chip is read again around it; Auto-center offers a first guess (mean shift to the nearest patch standing out from the chip's background, numpy only). Import to project makes one label per labelled chip, at its centre with lon/lat, attribution and a description of what the model said, in one undo step, and notes every verdict on the chip's record in the review file (3.11); ignored chips make no label. Chips the file notes as reviewed are kept off the grid (Show reviewed brings them back, shaded); one given a new verdict has its label changed by the next import, removed for Ignore, or made if it was ignored, and one re-centred has its label moved; one whose label is gone is plain again. A chip within Labelled within of a label already on its image is already in project: framed in blue, on the grid unless Show already in project is unticked, its labels marked in Details; its verdict goes onto the nearest, or makes a new label beside it when Details' tick says so; a chip with a verdict in hand is listed whatever the boxes say. A Labels panel on the left of the grid stacks Ignore and every class with its colour, its key (1-9, then Shift+1-9) and the count of chips an import gives it, a click making it the active label; a filter box shortens the list and Enter takes the first; a class dragged up or down moves under another key, Ignore staying on top, the order kept by name in the user's settings while the project's list and the colours stay as they are. A Details panel beside the grid shows the chip in hand - under the cursor, last clicked or under the mouse - read at a few times the chip size with a box where it is, and everything known of it, a Measure tool (R: two clicks on the picture give the length between them in metres by the georeferencing, pixels alone without), with a Reference strip of chips (one to six to a row, as its head says; two by default) already given a chosen class under it; Show on image (M) opens the whole image, decimated, in a window with a box where the chip is; Show lists all chips, the ones still to look at, the labelled - all, or one class - or the ignored, or one standing with its count: reviewed before, changed (an earlier answer with a new verdict), already in project - those three whatever the Show boxes say, greyed at none; Score at least clips it to the chips scored at least that. Train on verdicts (source runs with PyTorch, requirements-ml.txt; the installed build leaves it out) teaches a classifier - a small net from scratch, or a linear head on frozen ImageNet ResNet18 features (torchvision) - the verdicts, the file's notes and the project's labels of those classes, Ignore as a class, by steps with class-weighted cross-entropy; from thirty chips a fifth is held out and the accuracy reported on it. Chips are read once with each image opened once, scaled to 64 px and saved under the system's temporary folder (one .npz per image, chip size and display setting), and read from there after. A training window shows the progress, then the report per class, the confusion matrix and the curves. Every chip left gets a suggested class and confidence; the grid orders by it, S accepts a suggestion, Accept the page takes those at or above a confidence, Suggest runs the model again without training.
Display Settings	View > Display Settings... (Ctrl+D), or a group's right-click menu, sets how that group's imagery is drawn: through the file's own colour table when a band carries one (the Colour table tick turns that off; a Legend row lists the values present, each with its colour and share), one band as grayscale, or a band for each of red, green and blue, with brightness, contrast and gamma per channel over a histogram of the band; in red/green/blue mode a channel can be hidden, drawing it as black. Auto spreads a band's 2-98 percentile range from black to white. For 16-bit and floating-point imagery the window also sets which values are drawn black and white: percentiles of each band (2 to 98 by default), or one fixed range of values for every band - 0 to 1 for imagery that is already a fraction, which the window suggests when the sampled values lie within it. Changes show on the canvas, in the Snippet Panel and in the Snippet Editor as they are made; OK keeps them, Cancel puts back what was there. Sub-groups inherit a group's settings unless they set their own. Saved in the project (3.10). Display only: exports keep the raw pixels.
Pixel values	The status bar shows, beside the cursor's coordinates, the pixel under it: every band's value, nodata named, and through a colour table the value's colour. View > Pixel Values in Status Bar turns it off; the choice is remembered.

Overlay	Effect
Who did what	Each person enters a user name - asked on first start, changed with File > Switch User..., shown in the status bar; no password. Labels record who created them and who last set each value, and each mask who drew it (3.3, attribution); the marker's tooltip says who labelled it. Undo takes an attribution back with its value. Import Ground Truth and Combine Projects keep the attribution of the labels they bring in. The groundwork for several people labelling the same objects and a reviewer comparing them.
Import Options	File > Import Options sets what becomes of a label brought in - by Import Ground Truth, Combine Projects or a Model Review import - that lands within a chosen number of image pixels of a label already here on the same image: keep the labels already here and drop the new (the default), keep the new and drop the old, keep both, or ask - a window deciding each pair by hand, each new label paired with the nearest label already there. Settings, not the project. What is dropped is part of the import's undo step; Combine Projects writes a new file and is not undoable.
Re-seat Labels to Imagery	File > Re-seat Labels to Imagery reads the header of every labelled image and, where the file's size differs from the project's record (cropped since it was labelled), moves the labels and masks by the whole-pixel offset the georeferencing gives, drops labels now off the image, and records the new size and affine; where the pixel size changed, after a question, it places labels by latitude and longitude and drops their masks. An image without labels has its record brought up to date, nothing to move and no question. One undo step. After a project opens the same check runs over the labelled images (a metadata-only load opens no file) and offers the re-seat; an accepted re-seat is followed by a sweep of every image's header in the background that brings the records of unlabelled images up to date, and an image shown later is checked then too; the Model Review memory's chip centres move with their image; Locate Missing Images accepts a cropped copy of an image (the same grid, lying within the recorded extent) as that image.
Remove Missing Images	File > Remove Missing Images lists the project's images whose files are not found and that are not open on the canvas, each with its label count; the ticked ones are removed from the project together with their labels. The Locate Missing Images dialog offers it too (Remove From Project). It is one undo step: the images return to their places with their labels and links. The dialog warns when every image in the project is missing, which usually means a drive or share is not connected. An image whose file went missing while it was open is removed from the layer panel instead.
Open Recovery	A recovery copy of the project is written as the user works (every minute, and soon after an edit) beside the application's settings, and offered back when a session ended unexpectedly. Declining the offer sets that copy aside under a dated name (recovery-YYYYMMDD-HHMMSS.geolabel), the last five kept, where the next autosave cannot overwrite it; File > Open Recovery opens the live copy or a kept one as an unsaved session, with no file of its own until saved.
Labels of the image shown	In Cycle, View Cycle and Labeled Cycle every label on the image the cycle is showing wears the same white corner brackets; labels of other images, which lie over it where images overlap, do not. The brackets sit outside the linked-group ring, dim with a rejected label, follow each step, and go when the cycle is left. Display only: nothing is stored.
Move a label	Right-click a label > Move, then click where it should be; right-click or Escape cancels. On its own image only its pixel position and lon/lat change - masks, orientation, size, rating and review stay - and where images overlap a click on the label's own image stays on it. A click on another image asks first (Move or Cancel): there the label joins that image, loses its masks, and keeps its true-north heading as that image's pixel angle where both are georeferenced (otherwise its orientation is cleared). The question can be switched off in the box itself or with Labels > Warn Before Moving a Label to Another Image; it is on until switched off. One undo step. The mover is recorded under attribution key position. The Snippet Editor's Move label tool makes the same move on the label's own image, from a click on its snippet.
Sections that open and close	The four sections left of the canvas - Layers, Labeled Images, Waypoints and Hard Negatives - each sit under a header with a chevron, down when open and right when closed. Clicking the header closes the section to that one row and gives its room to the sections still open; clicking again opens it at the size it had. The Waypoints header keeps Show on map in reach while closed. Which sections are closed is remembered between sessions.

Overlay	Effect
Undo and redo	Edit > Undo (Ctrl+Z) and Redo (Ctrl+Y or Ctrl+Shift+Z) step back and forward through the edits of this session: placing, removing, linking, unlinking and describing labels, group IDs, hard-negative flags, Clear All Labels, removing a class, Import Ground Truth, the Snippet Editor's masks (one step per stroke), orientation, ratings, reviews, notes, moves and sizes, the class, description and mask-name lists, and waypoints. The menu names the step it would take. A gesture that touches several labels - a box link, a heading or size shared to linked labels, a page accepted in the review grid - is one step. The Snippet Editor's Ctrl+Z and Ctrl+Y use the same history. Up to 200 steps are kept; saving keeps them, opening or starting a project clears them, and undoing back to the saved state clears the unsaved mark. Adding or removing imagery, relocating images and display settings are not undoable, and removing imagery drops the steps that touched it - except Remove Missing Images, which is undoable, since nothing of those images was on the canvas. The history is not saved with the project.
About	Help > About lists what the build is made of: the GeoLabeller version and whether it is the installed build or a source checkout, Python and the operating system, the libraries it runs on with the native libraries they wrap (PyQt5 and Qt, numpy, rasterio with its GDAL and PROJ, pyproj with its PROJ, h5py with HDF5, Pillow, affine), the install folder and settings file, and every bundled package with its version. Copy puts the whole page on the clipboard as text, for a bug report.
Waypoint export and import	Export > Waypoints (GPX)... writes every waypoint to a GPX 1.1 file; File > Import Waypoints... adds the waypoints in a GPX file, or in another GeoLabeller project, that are not already in the project (section 3.9).
Label export (KML)	Export > Labels (KML)... writes every label with ground to a KML 2.2 file: a Folder and a Style per class in its marker colour, the label's particulars as ExtendedData (section 3.12).
Rename Class	Labels > Rename Class... gives a class a new name on every label that has it, on every image, and in the Model Review memory's became field (predicted, the model's own word, is left); the class keeps its place in the class list and so its colour. A name the project has already merges the two classes under it, after a question with the counts: the old name leaves the list. One undo step. Edit Classes still reads a changed line as a removal and an addition, which deletes the labels.
Clean Close Labels	Labels > Clean Close Labels... lists, on every image, each two labels whose centres lie within a distance in pixels of the image (the Import Options distance to start with), filtered to any classes, one class only or different classes, with or without pairs that share an object_id, and shows them one at a time with both snippets; Keep both, Remove left, Remove right or Remove both. A removal is made at once as one undo step, and every pair involving a removed label is dropped. The survivor keeps its object_id and links. The search buckets each image's labels by a grid of the distance (one pass, not every label against every other).
Label Markers	View > Label Markers chooses how the labels sharing an object_id are drawn on the canvas: every label as its own marker; one marker per object (its caption counting the views) with the other labels as small dots, still hit-testable; or one marker per object alone, the others not drawn and not hit-testable. The marker drawn is the label on the image a cycle is showing, else the one a reveal from a panel asked for, else the one on the topmost shown layer, else the lowest id. Display only: the project is unchanged. A setting (view/label_markers).
Hover Snippet	View > Hover Snippet (hold Ctrl): with Ctrl held and the mouse at rest on a marker for about a third of a second, a small window beside the cursor shows the snippet around that label at the size chosen (96 to 256 source pixels, shown at that size; or Off), read through the group's Display Settings, captioned with class, image and description. It closes when the mouse leaves the marker, Ctrl is released, or a button or the wheel is used. A setting (view/hover_snippet_px).
Snippet Editor: one object	The list's Class dropdown offers, after All classes and each class, every linked object - named by its class (classes, when its labels disagree), its group ID when it has one, the start of its id and how many labels it has. Choosing one lists only that object's labels, across every image, with the Show filter and both views working as usual; the choice is kept while the list is refreshed, and falls back to All classes when the object is gone.
Snippet Editor: linked row	Under the Single view, when the snippet in hand belongs to a linked object, a row shows every snippet of that object side by side, captioned with its image, class and the list's badges (ratings, headings, sizes, reviews) for the sections switched on; the heading counts the object's images and notes when its labels disagree on class. Clicking a snippet in the row makes it the one in hand - switching the list to the object, and the Show filter to All, when the list was not showing it. The row is hidden for a label that is not linked.

Overlay	Effect
Change a label's class	The Snippet Editor's Label class picker (Ctrl+1-9 in the Single view picks by place in the class list) gives the snippet in hand another class; typing a name that is not a class yet and pressing Enter creates it in the project. With Also change linked labels ticked (the default, remembered) every label linked to it takes the class too; unticked, only that snippet changes - for the part of an object that needs a label of its own. The canvas's right-click Change Class menu changes the whole object. The change is in the project, so every later save and ground truth export carries it; one undo step, which also removes a class it created. When the editor is showing one class and the change takes the snippet out of it, the editor moves on to the next snippet.
Snippet Editor: Review	The Review section marks each snippet accepted, rejected or to be looked at again, with buttons or, in the Single view, the keys A, X and F; with Advance after marking on (the default) the editor moves to the next snippet. The worklist is Needs review (unreviewed or to be looked at again) against Reviewed. A rejected label stays in the project and the ground truth file, is drawn dimmed on the canvas, and is left out of the HDF5 and sub-image exports, whose summaries count it. Each mark is one undo step. The Grid view has a Review mode beside Orient (the choice is remembered) for marking a page at a time: each snippet wears what its review found - a green, red or amber border, a rejected one dimmed and struck through; a click rejects a snippet and a second click takes that back, a right-click flags it to look at again, and a double-click still opens it in the Single view, marking nothing. Accept the rest of the page (key A) accepts every snippet on the page left unmarked, as one undo step, and goes on to the next page.
Snippet Editor: zoom and the label point	In the Single view a zoom set with the wheel stays for the snippets that follow, with the view held on the label - the next label's own point lands where the last one's was - until Fit, beside the zoom readout, goes back to the whole snippet; undo, redo and the brightness and contrast sliders redraw the same view. Changing the snippet size starts fitted. The label's own point is marked on the snippet with a ring (near an image's edge it is not the middle of the snippet); Label point unticks it, and the choice is remembered.
Snippet Editor: Move label	The Move label tool (key V) moves the label to where its snippet is clicked, on its own image, to a fraction of a pixel when zoomed: its pixel position and lon/lat change and nothing else, as with the canvas's Move. One undo step; recorded under attribution key position. The tool stays in hand, so a pass correcting positions is Space and a click per snippet; it is never the tool the editor opens with.
Snippet Editor: Notes	The Notes section shows the description (3.3) and the group ID of the snippet in hand and edits them in place. The description is that label's alone; the group ID is given to every label linked to it. A note is stored when its field is left - a click elsewhere, Ctrl+Enter in the description, Enter in the group ID - and Escape puts back what was stored. Each is one undo step, the same as from the canvas's right-click menu. The section has no pair on the Show filter.
Snippet Editor	Labels > Snippet Editor... is one window for everything recorded per snippet; in 2.0 it replaced the separate Orientation Editor and Mask Editor windows. A list of the project's snippets - one class or all, in class, image and label order, with a Show filter offering 'needs' and 'done' for each section - feeds two views: Single, one snippet at a time and zoomable, with the Orient, Paint masks, Polygon, Wand, Measure and Move label tools (keys O, P, G, D, M and V); and Grid, a page of snippets at once, where double-clicking one opens it in the Single view. A column of sections - Notes, Orientation, Masks, Size, Confidence, Review - each switches on and off, taking its panel, filter pair, overlay, tool and keys with it. Everything is written to the project as it is done; the save bar says whether it is on disk yet (Ctrl+S saves). The list reads thumbnails for the rows in view, and the Grid view is built while it is showing, so the editor opens and follows an undo quickly on projects of thousands of labels.
Orient	In the Snippet Editor's Grid view, or with the Orient tool in its Single view, dragging tail-to-nose across a snippet stores orientation_px_rad and, for georeferenced imagery, orientation_deg; right-click clears it. Snippets are raw source pixels with the export's framing, so the drawn vector is a source-pixel vector. Oriented snippets get a coloured border: amber for hand-drawn, violet for propagated. With the 'apply heading to linked labels' toggle on, drawing on a linked label gives the group's other labels the same true-north heading, each with a pixel angle derived through its own image (marked orientation_derived until drawn over by hand).

Overlay	Effect
Mask	In the Snippet Editor's Single view, the Paint masks tool paints named binary masks on a label's snippet (left-drag paints, right-drag erases). The mask to paint is chosen by an active mask name, picked from or typed into the project's mask_names list (3.1); Add Mask creates it and selects it in one click. Allow Overlap (off by default, remembered per user) decides whether a stroke may cover another mask's pixels; while it is off, other masks also act as edges for Fill Enclosed. The snippet size is selectable; masks are stored anchored to the full source image, so resizing changes only the window being painted - existing masks are never moved or cropped by it. The wheel zooms to the cursor (0.25x-32x, pixels stay crisp; the zoom stays for the next snippet until Fit) and Shift+drag pans, so single-pixel edits are practical. Space / Ctrl+Space step through the snippet list, whose Show filter offers the snippets that need masks or already have them. Fill Enclosed floods the interior of a drawn outline, and refuses when the outline has a gap (nothing enclosed). Brightness and contrast sliders change the display only. A live readout compares the object's RAW per-band mean/std against the background. Stored per 3.8; carried into the HDF5 export automatically.
Mask tools	Beside the brush (key P; [and] make it smaller and larger, by about a fifth of its size) two more tools make masks. Polygon (key G): a left click begins a shape to fill into the active mask, a right click one to cut out of it; further left clicks add corners, a right click or Backspace takes the last one back, and Escape gives the shape up; a double-click, Enter or a click on the first corner closes it. A pixel is in the shape when its centre is. Wand (key D): a click takes into the active mask the connected pixels that look like the one clicked - every channel within Tolerance (0-255, in display levels) of it, judged on the snippet as it is displayed, brightness and contrast included - and a right-click takes them out; holding the button and dragging right or left raises or lowers the tolerance, the patch being shown before it is taken. Grow and Shrink make the active mask one pixel wider or narrower all round, and Remove Specks drops its separate pieces smaller than one stamp of the brush (the largest piece always stays); with Fill Enclosed these tidy what the wand took. Holding H leaves the masks undrawn, to see the imagery under them. Each of these is a mask edit like a stroke: stopped by the other masks unless Allow Overlap is on, stored as any mask is (3.8), and one undo step. No segmentation model is involved: the wand compares pixel values.
Confidence	In the Snippet Editor's Confidence section, the buttons - or keys 1-5 in the Single view - rate a snippet from 1 (lowest) to 5 (highest); 0 clears the rating. With Advance after rating ticked (off until ticked, remembered) a rating key also moves to the next snippet. Stored as the label's confidence (3.3), omitted when unrated, and exported to HDF5 as the per-sample confidence dataset, 0 for unrated labels and for negatives (3.8).
Group location	Right-click a group > Set Location... tags every image in the group (sub-groups included) with a free-text location such as "New York Area"; one undo step. The tag is stored per image (3.2), so it survives regrouping, and every HDF5 sample cut from a tagged image records it in the per-sample locations dataset (aligned with images; absent when nothing is tagged). An empty entry clears the tag.
Remove a layer or group	Right click in the layer panel. This DELETES the images and their labels from the project, not just from the view - including, for a group, images the project files under it that the panel never showed, such as a shared project's images that are not on this machine. The confirmation states how many of each, and a single unlabelled image is removed without asking. Hiding without deleting is the visibility checkbox.
Flag hard negatives	Right click an image to mark it as a hard-negative source: it holds confusers but no true positives. Flagged images collect in the Hard Negatives panel, and the H5 export can take its hard negatives from them alone.
Export one object	Right click a label on the canvas, or an "Object: ..." row in the labelled panel, > Export Object Snippets... The HDF5 export opens scoped to that linked object: its true-positive snippets from every image it appears in, or only from the layers toggled on. Instances keep their own class names, so a linked object whose views disagree exports as it was labelled - the dialog shows the breakdown first.
Choosing what an export contains	The HDF5 dialog asks two independent questions: which images contribute gt=true examples (a selected object, the layers toggled on, all loaded layers, or nothing) and which get the hard-negative window slid over them (nothing, the same images as the examples, the layers toggled on, flagged sources only, or all loaded layers). Labelled ground is never written as a hard negative whichever images are slid.

6.2 How a label's fields are produced

Every field in 3.3 is written for every label; these are the ones whose value depends on how or where the label was made.

Field	Produced by	Consumer note
pixel_x, pixel_y	Any left click in a labelling mode	Always the source image's own pixels at full resolution, whatever the zoom or pyramid level on screen; a click that falls outside them - the transparent margin a reprojected raster leaves inside its bounding box - places no label at all. Stored as fractions - see 3.4.
lon, lat	Derived from the clicked pixel through the image's CRS	Zero when the image has no CRS. Check crs_epsg (or crs_wkt) on the image before trusting these.
object_id	Shared when labels are linked	Unique per label until then. Labels of one real object are found by grouping on this value, not by proximity.
length_m, width_m	Snippet Editor, Size section	Absent when never measured; either may be present without the other. Geodesic metres, not planar. Copied to every label sharing the object_id when 'Apply size to linked labels' is on.
description	Right click a label, Description...	Free text, absent when empty. Per label even when labels are linked: the same object in two images is described separately.
group_id	Right click a label, Group ID...	One value for the whole linked group, always: setting it on any member applies it to every label sharing the object_id. A label that is unlinked loses it; a label joining a named group adopts the name (the first label's name wins a merge conflict).
class_name	The class selected when the click was made	Always one of the top-level classes.

6.3 Keyboard and mouse reference

Generated from the application's own binding table, so it matches the reference shown by F1 exactly.

File Operations	
Ctrl+N	New Project
Ctrl+Shift+P	Open Project
Ctrl+S	Save Project
Ctrl+Shift+S	Save Project As
Ctrl+O	Add Image (GeoTIFF + custom)
Ctrl+Shift+O	Add Directory
Ctrl+Q	Exit

Edit	
Ctrl+Z	Undo the last edit (labels, flags, presets, waypoints)
Ctrl+Y	Redo
Ctrl+Shift+Z	Redo

Navigation	
Mouse Wheel	Zoom in/out
Click + Drag	Pan (in Pan mode)
Right-click	Context menu
Ctrl+G	Go to a latitude/longitude
Ctrl+Shift+W	Add waypoint by coordinates
Ctrl+D	Display Settings for the selected group
Ctrl+B	Show/hide the snippet sidebar

Mode Switching	
P	Pan mode
L	Label mode
C	Cycle mode (group-based)
V	View Cycle (the selected group's images on screen)
Shift+C	Labeled Cycle (the selected group's images that have labels)
W	Waterfall mode (stack group vertically)
R	Ruler mode

Labeling (Label & Cycle modes)	
Left-click	Place label
Right-click label	Label options (remove, link, describe)
Right-click + drag	Pan the view
Ctrl+Left-click label	Label options (Cycle modes)
1-9	Quick-switch to class 1-9
Shift+1-9	Description preset 1-9; again for none
Shift+0	No description (new labels get none)
Right-click label > Move	Then click its new position
Escape	Cancel link mode, or a move

Chain Linking	
K	Toggle chain-link (any labeling mode)
Left-click label	Add to chain (first click anchors)
N	Finish this chain, start a new one
Escape	Exit chain-link (links are kept)

Cycle Modes (Cycle / View Cycle /	
Space	Next image (hides the one the cycle showed)
Ctrl+Space	Go back to previous layer
Left-click	Place label
Right-click label	Label options (remove, link, describe)
Right-click + drag	Pan around
Mouse wheel	Zoom in/out

Waterfall Mode	
Hold Space	Glide up through the image stack
Hold Ctrl+Space	Glide down (starts at the bottom)
+ / -	Glide speed up / down
Left-click	Place label (group labels stay visible)
Right-click label	Label options (remove, link, describe)
Right-click + drag	Pan within the stack
Mouse wheel	Zoom in/out

Measuring	
Shift + Drag	Measure distance, without changing mode
Labels > Snippet Editor	A label's length and width (Size)

Measuring	
Escape	Clear the measurement or the go-to marker

Waypoints	
Right-click map	Add waypoint here (Pan mode, geo)
Ctrl+Shift+W	Add waypoint by coordinate
Right-click marker	Go to / Rename / Remove
Double-click in panel	Fly to that waypoint
Show on map	Hide or show every waypoint marker
Export > Waypoints (GPX)...	Write every waypoint to a GPX file
Export > Labels (KML)...	Write every label with ground to a KML file
File > Import Waypoints...	Add the waypoints in a GPX file or another project

Layer Panel	
Checkbox	Toggle layer/group visibility
Right-click group	Select/Unselect, Expand/Collapse all
Right-click layer	Zoom to layer, Remove (deletes from project)
Drag & Drop	Reorder layers/groups

Labeled Images Panel	
Checkbox	Toggle image visibility (synced)
Right-click label	Zoom to label or layer
Right-click group	Select/Unselect all in group

Snippet Editor	
Space	Next snippet (Single view)
Ctrl+Space	Previous snippet (Single view)
1-5	Rate confidence, 1 lowest; then next, if Advance is ticked
0	Clear the confidence rating
A	Review: accept, then next (Single view)
X	Review: reject, then next
F	Review: look at again, then next
Ctrl+1-9	Label class by its place in the list (whole object)
O / P / G / D / M / V	Tool: Orient / Paint masks / Polygon / Wand / Measure / Move label
Click (Wand)	Take the connected pixels that look like this one
Right-click (Wand)	Take them out of the mask
Drag (Wand)	Right for more, left for less, before letting go
Click (Polygon)	Add a corner; double-click or Enter fills it
Right-click (Polygon)	First click: cut the shape out. After: take back a corner
Backspace / Escape	Polygon: take back a corner / give it up
Left-drag	Paint a mask, draw a heading, or measure
[/]	Brush smaller / larger
H (hold)	Look under the masks
Click (Move label)	Put the label there, on its own image
Right-drag	Erase the mask (Paint)
Right-click	Clear the heading (Orient) or the size (Measure)

Snippet Editor	
L / W	Measure: the next line is the length / the width
Ctrl+Enter	Notes: store the description being typed
Enter	Notes: store the Group ID being typed
Escape	Notes: put back what was stored
Shift+drag	Pan the zoomed snippet
Mouse wheel	Zoom to the cursor; kept until Fit
Double-click	Open a grid cell in the Single view
Click (Grid, Review)	Reject the snippet; again takes it back
Right-click (Grid, Review)	Flag the snippet to look at again
A (Grid, Review)	Accept the rest of the page, then the next
Ctrl+Z	Undo - one history with the main window
Ctrl+Y	Redo
Ctrl+S	Save Project

Keeping Labels Tidy	
Labels > Rename Class...	A new name for a class on every label that has it; a name already there merges the two
Labels > Clean Close Labels...	Every two labels within a distance on one image, side by side: keep both, remove one, or both
Left / Right	Clean Close Labels: the previous / next pair
K	Clean Close Labels: keep both, then the next pair
1 / 2 / 3	Clean Close Labels: remove the left / the right / both (one undo step each)
View > Label Markers	A linked object as one marker (counting its views), the other views as dots or not drawn; or every label
Ctrl + rest on a label	Its snippet beside the cursor (View > Hover Snippet sets the size, or Off)

Find Candidates (Labels > Find C	
Labels > Find Candidates...	A detector proposes places that stand out, to check in Model Review
Check against my labels	What each sensitivity would have found of the labels, and how many candidates an image gets
Run	Write the candidates file beside the project and open it in Model Review
Double-click a row	Take that sensitivity

Model Review (Labels > Model Re	
Ctrl+O	Open a file of the model's mistakes
Left-click chip	Give it the active label
Labels panel (left)	Click a class: the active label, onto the picked chips or the cursor's; type in its box to shorten the list, Enter takes the first
Drag a class (Labels panel)	Put it under the key you want; Ignore stays on top, and the order is yours from then on
Shift+Left-click	Label every chip from the last click to this
Ctrl+Left-click	Pick a chip without labelling; a key then labels the picked
1-9	The class at that place in the list: onto the picked chips, or the chip under the cursor (then the next)
Shift+1-9	The tenth to the eighteenth class, the same way
Show: list	All, To look at, Labelled, Ignored; Reviewed before, Changed, Already in project - those three whatever the Show boxes say; one entry per labelled class
0	Ignore: looked at, nothing to label

Model Review (Labels > Model Re	
Arrows / Home / End	Move the cursor from chip to chip; past the page's end it turns the page
Enter	The active label onto the chip under the cursor, then the next
Space	The next chip, this one left as it is
I	Ignore the rest of the page, then the next page
R	Measure: two clicks on the Details picture give the length between them, in metres by the image's georeferencing and in pixels (pixels alone without); a right-click clears
M	Show the chip in hand on its whole image, in a window: the wheel zooms, a drag pans, F fits, Escape closes
Train on verdicts button	A model learned from the verdicts suggests a class for the chips left; Suggest runs it again; Save snippets reads the chips ahead (source runs with PyTorch, requirements-ml.txt)
S	Accept the suggested class: onto the picked chips, or the chip under the cursor (then the next)
Double-click chip	Put the label's centre there, on the object (it does not label)
Auto-center button	Move the picked chips' centres (or the page's) onto what stands out
Right-click / Delete	Take a verdict back, and the centre with it
Escape / Ctrl+A	Pick none / every chip on the page
PageDown / PageUp	Next / previous page of chips
Show already in project	Chips with a label on their image within Labelled within (blue frame); Details marks the labels, and ticks a new label beside them
Show reviewed	Chips the file notes as imported or ignored, shaded
Ctrl+Enter	Import to project: the labels, and each chip's note into the file
Ctrl+Z	Take back the last thing done here - a verdict, a run, a centre move; then undo in the main window's history
Ctrl+Y	Redo, likewise
Ctrl+S	Save Project

Help	
F1	Show this help